

Relatório Técnico

Sistema Inteligente de Perguntas e Respostas no Estilo Akinator

Identificação de Personagens de Anime e Mangá por meio de um Sistema Baseado em Conhecimento

Questão 2.1 — Sistema Inteligente no Estilo Akinator

1. Introdução

Este relatório apresenta o desenvolvimento de um sistema baseado em conhecimento inspirado no funcionamento do Akinator, elaborado como resposta à Questão 2.1. O objetivo do trabalho é construir um sistema capaz de identificar, por meio de uma sequência de perguntas respondidas com "Sim", "Não" ou "Não sei", um elemento pertencente a um domínio previamente definido, reduzindo progressivamente o conjunto de hipóteses possíveis até convergir para a resposta mais provável.

O domínio escolhido foi o de personagens de anime e mangá, enquadrando-se na categoria "personagens de séries ou desenhos" sugerida no enunciado. O sistema foi implementado em Python, com interface interativa construída na biblioteca Streamlit, e utiliza arquivos JSON tanto para armazenar a base de conhecimento (personagens.json) quanto para registrar o histórico de testes automáticos executados sobre o sistema (testes.json).

As seções seguintes detalham, respectivamente, a descrição do domínio escolhido, a estratégia adotada para representar o conhecimento, o mecanismo de inferência utilizado para eliminar hipóteses e decidir a resposta final, exemplos reais de interação gerados a partir do próprio algoritmo do sistema, e uma análise crítica dos resultados obtidos nos experimentos automáticos registrados em testes.json.

2. Descrição do Domínio

O domínio escolhido para o sistema é o universo de personagens de anime e mangá. Trata-se de um domínio ficcional amplamente reconhecido pelo público de teste, o que facilita a coleta de respostas confiáveis durante os experimentos, e ao mesmo tempo suficientemente rico em arquétipos — protagonistas, vilões, anti-heróis, magos, guerreiros, ninjas, piratas, caçadores, entre outros — para sustentar perguntas de "Sim/Não" genuinamente distintas entre os candidatos.

A base de conhecimento atual reúne 27 personagens, provenientes de 15 obras distintas, o que excede o mínimo de 20 entidades exigido no enunciado. A Tabela 1 lista os personagens cadastrados e a obra à qual pertencem. Cada personagem é caracterizado por 20 atributos binários (seção 3), também acima do mínimo de 15 atributos exigido.

Para reduzir ambiguidades decorrentes de personagens com múltiplas formas ou fases ao longo da obra (como transformações de Goku, Vegeta ou Naruto), a própria interface do sistema orienta o usuário a considerar "a versão pela qual o personagem é mais conhecido" ao responder às perguntas — uma decisão de projeto que ajuda a manter a consistência entre os atributos cadastrados e o que o usuário provavelmente tem em mente.

2.1 Base de personagens cadastrados

Personagem	Obra de origem
Naruto	Naruto
Sasuke	Naruto
Goku	Dragon Ball
Vegeta	Dragon Ball
Luffy	One Piece
Zoro	One Piece

Personagem	Obra de origem
Nami	One Piece
Light	Death Note
L	Death Note
Deku	My Hero Academia
Bakugo	My Hero Academia
Todoroki	My Hero Academia
Gojo	Jujutsu Kaisen
Yuji	Jujutsu Kaisen
Tanjiro	Demon Slayer
Nezuko	Demon Slayer
Levi	Attack on Titan
Eren	Attack on Titan
Mikasa	Attack on Titan
Saitama	One Punch Man
Anya	Spy x Family
Guts	Berserk
Gon	Hunter x Hunter
Killua	Hunter x Hunter
Ichigo	Bleach
Usagi Tsukino	Sailor Moon
Jotaro Kujo	JoJo's Bizarre Adventure

Tabela 1 — Personagens cadastrados na base de conhecimento (27 personagens, 15 obras)

Como se observa na tabela, os personagens não estão distribuídos uniformemente entre as obras: séries com elenco mais amplo — "My Hero Academia", "Naruto", "One Piece", "Death Note", "Jujutsu Kaisen", "Demon Slayer", "Attack on Titan" e "Hunter x Hunter" — contribuem com dois ou mais personagens cada, o que é também a origem das situações de maior ambiguidade entre hipóteses, discutidas na seção 6.

3. Estratégia de Representação do Conhecimento

O conhecimento do sistema é representado de forma explícita e simbólica em um arquivo de fatos no formato JSON (personagens.json). Cada personagem corresponde a um registro (um "quadro" de atributo-valor) contendo seu nome, a obra de origem e um conjunto fixo de 20 atributos binários, cujos valores possíveis são restritos a um vocabulário controlado de três símbolos: "s" (sim), "n" (não) e, reservado para uso durante a interação, "?" (não sei / não informado).

Essa é uma representação do tipo atributo-valor (representação vetorial de características), equivalente a uma tabela de fatos em que cada linha é uma hipótese do espaço de busca e cada coluna é uma pergunta que o sistema pode fazer para testá-la. A verificação de consistência dos dados confirma que os 27 personagens compartilham exatamente o mesmo esquema de 20 chaves, sem valores ausentes ou malformados — propriedade importante, pois qualquer atributo faltante seria silenciosamente tratado como "não sei" pelo motor de inferência (personagem.get(atributo, "?")), o que poderia distorcer tanto a filtragem de hipóteses quanto o cálculo de pontuação.

Um segundo artefato de conhecimento, complementar ao anterior, é o dicionário PERGUNTAS, que mapeia cada chave interna de atributo (ex.: "ninja") ao seu enunciado em linguagem natural correspondente ("É ninja?"). Esse dicionário é a camada explícita de tradução entre a representação interna, manipulada pelo algoritmo, e a interação com o usuário — não há regras implícitas ou codificadas apenas na lógica do programa: o significado de cada atributo está documentado e é diretamente exibido ao usuário.

3.1 Atributos utilizados

Atributo (chave interna)	Pergunta apresentada ao usuário
humano	O personagem é humano?
poder	Possui poderes especiais?
pirata	É pirata?
ninja	É ninja?
estudante	É estudante?
protagonista	É protagonista?
espada	Usa espada?
vilao	É vilão?
masculino	É masculino?
alien	É alienígena?
professor	É professor?
detetive	É detetive?
guerreiro	É guerreiro?
magico	Utiliza magia?
anime_moderno	É de anime recente?
crianca	É criança ou adolescente?
cacador	É um caçador?
transformacao	Possui transformação marcante?
olhos_especiais	Possui olhos especiais?
anti_heroi	É um anti-herói?

Tabela 2 — Atributos utilizados e pergunta correspondente

É importante observar que os 20 atributos não têm o mesmo poder discriminativo. A Tabela 3 contrasta os atributos mais equilibrados — cuja divisão entre "sim" e "não" na base é mais próxima de 50/50 — com os mais raros, concentrados em pouquíssimos personagens.

3.2 Poder discriminativo dos atributos

Atributo	Personagens "sim"	Personagens "não"	Diferença sim – não
protagonista	15	12	3
crianca	12	15	3
transformacao	12	15	3

Atributo	Personagens "sim"	Personagens "não"	Diferença sim – não
vilao	1	26	25
professor	1	26	25
detetive	1	26	25

Tabela 3 — Exemplos de atributos equilibrados versus atributos raros

Atributos equilibrados, como "protagonista" (15 sim / 12 não), são estatisticamente os melhores candidatos para perguntas iniciais, pois qualquer que seja a resposta, aproximadamente metade da base é eliminada. Já atributos raros, como "vilao" (1 sim / 26 não), têm pouco valor no início do jogo, mas tornam-se decisivos quando o conjunto de candidatos já foi reduzido a poucos nomes que compartilham quase tudo entre si. Essa variação de "poder de corte" entre atributos é justamente o que o mecanismo de inferência, descrito na seção 4, explora de forma automática e dinâmica.

4. Mecanismo de Inferência Utilizado

O mecanismo de inferência adotado baseia-se principalmente na busca em espaço de hipóteses, em que cada um dos 27 personagens cadastrados representa uma hipótese inicial igualmente válida. A cada resposta fornecida pelo usuário, hipóteses incompatíveis são eliminadas por meio de regras de comparação entre os atributos do personagem e a resposta informada. Quando a eliminação por si só não é suficiente para isolar um único candidato, o sistema recorre a um critério de pontuação para ordenar as hipóteses restantes, caracterizando uma abordagem híbrida que combina busca em espaço de hipóteses, regras de inferência simples e um componente de ranqueamento por similaridade.

Essa abordagem híbrida é implementada por meio de três funções centrais do código-fonte, descritas a seguir.

4.1 Seleção da pergunta — busca gulosa em espaço de hipóteses

A função `melhor_pergunta()` implementa a etapa de busca propriamente dita. A cada rodada, para cada atributo ainda não perguntado, o sistema conta quantos dos candidatos restantes respondem "sim" e quantos respondem "não" àquele atributo. Atributos em que todos os candidatos concordam (contagem "sim" ou "não" igual a zero) são descartados por não terem poder de eliminação naquele momento. Entre os atributos restantes, o sistema escolhe aquele cuja diferença absoluta entre as contagens de "sim" e "não" é a menor possível — ou seja, a pergunta cuja divisão do conjunto de candidatos é a mais equilibrada.

Esse critério é uma heurística gulosa (greedy), podendo ser entendida como uma versão simplificada do ganho de informação usado na indução de árvores de decisão (como no algoritmo ID3): em vez de calcular a entropia exata de cada partição, o sistema usa o desequilíbrio $|sim - não|$ como substituto direto e computacionalmente mais simples para "quanto essa pergunta deve reduzir o espaço de hipóteses, na pior das hipóteses". A melhor pergunta é recalculada dinamicamente a cada rodada, a partir do conjunto de candidatos efetivamente restante — o que caracteriza uma busca on-line em espaço de hipóteses, e não uma árvore de decisão estática.

4.2 Eliminação de hipóteses — regras de comparação

A função `filtrar_candidatos()` implementa a eliminação: um personagem permanece candidato se, e somente se, o valor armazenado para o atributo perguntado for idêntico à resposta dada pelo usuário. Quando a resposta é "não sei", nenhuma regra de eliminação é aplicada e todos os candidatos são mantidos — decisão de projeto que prioriza não descartar erroneamente a hipótese correta em troca de progredir mais lentamente na redução do espaço de busca.

4.3 Pontuação e desempate — ranqueamento por similaridade

As funções `calcular_pontuacao()` e `ranking_candidatos()` implementam o componente de pontuação. Para cada candidato restante, a pontuação é o número de atributos, dentre os já respondidos (ignorando os "não sei"), cujo valor armazenado coincide com a resposta dada. Os candidatos são ordenados de forma decrescente por essa pontuação. Esse ranqueamento é usado em dois momentos: (i) para exibir ao usuário, a qualquer momento, a hipótese mais provável e o total de hipóteses restantes, pelo painel "Ver hipóteses atuais"; e (ii) como critério de decisão final, quando não há mais atributo capaz de dividir o conjunto de candidatos — nesse caso, o

personagem mais bem pontuado é apresentado como resposta, garantindo que o sistema sempre encerre com uma hipótese concreta.

4.4 Condições de encerramento

O laço de perguntas se encerra em uma de três situações: (1) restar apenas um candidato compatível com todas as respostas dadas, caso em que ele é apresentado imediatamente; (2) esgotarem-se os atributos ainda não perguntados capazes de discriminar os candidatos restantes, caso em que o ranqueamento por pontuação decide o palpite; ou (3) o conjunto de candidatos ficar vazio, caso em que o sistema informa não ter conseguido identificar o personagem e oferece ao usuário a opção de cadastrá-lo (modo de aprendizado, seção 4.5).

- INÍCIO: candidatos $\leftarrow$ todos os 27 personagens da base;
- Se candidatos está vazio $\rightarrow$ "Não consegui identificar" $\rightarrow$ entra em modo de aprendizado;
- Se restou 1 único candidato $\rightarrow$ sistema apresenta o único candidato restante;
- Caso contrário, seleciona a melhor pergunta (heurística: minimiza $|\text{sim} - \text{não}|$);
- Se não há pergunta útil restante $\rightarrow$ ranking por pontuação escolhe o candidato mais compatível;
- Caso contrário, pergunta ao usuário (Sim / Não / Não sei) e filtra os candidatos, repetindo o processo a cada resposta.

Observação técnica verificada experimentalmente: como `melhor_pergunta()` só propõe um atributo quando, entre os candidatos atuais, tanto a contagem de "sim" quanto a de "não" são maiores que zero, qualquer resposta do usuário a essa pergunta necessariamente preserva pelo menos um candidato real da base. Por indução, isso significa que, enquanto a base de conhecimento não estiver vazia, o conjunto de candidatos nunca chega a zero apenas respondendo Sim, Não ou Não sei — o que foi confirmado simulando fielmente o algoritmo com uma estratégia de respostas adversarial (sempre escolhendo o valor minoritário em cada pergunta): mesmo nesse cenário extremo, o sistema convergiu para 1 único candidato, nunca para 0. A implicação prática é que um personagem ausente da base não faz o sistema dizer "não sei" — ele faz o sistema "chutar com confiança" o personagem cadastrado mais parecido (ver Exemplo 3, seção 5, e seção 6).

4.5 Aprendizado incremental da base de conhecimento

Quando o palpite do sistema é marcado como incorreto pelo usuário (botão "✗ Errou") ou quando o conjunto de candidatos se esvazia, o sistema entra em modo de aprendizado: o usuário é convidado a ensinar o novo personagem, respondendo às mesmas 20 perguntas da base atual. A função `adicionar_personagem()` grava esse novo registro em `personagens.json`, expandindo permanentemente a base de conhecimento para partidas futuras. Trata-se de um mecanismo de aprendizado supervisionado incremental, dirigido pela interação com o usuário — e não de aprendizado automático de padrões —, mas que permite ao sistema evoluir e reduzir, ao longo do tempo, a taxa de falha por ausência de personagens na base.

5. Exemplos de Interação

A interação com o usuário ocorre por meio de uma interface web em Streamlit, organizada em duas abas: "🎮 Jogo", onde a partida acontece, e "📊 Testes", onde os experimentos automáticos são executados e analisados (seção 6). Durante o jogo, o sistema apresenta uma pergunta por vez, com três botões de resposta — "✅ Sim", "✗ Não" e "🤔 Não sei" —, mapeados internamente para "s", "n" e "?". Uma barra de progresso indica quantas das 20 perguntas cadastradas já foram feitas, e um painel expansível, "🔍 Ver hipóteses atuais", permite consultar, a qualquer momento, a hipótese mais provável, o número de hipóteses restantes e as cinco mais bem pontuadas — com aviso explícito quando duas ou mais hipóteses estão empatadas na primeira posição. Ao final, o usuário confirma se o sistema acertou ("✅ Acertei") ou errou ("✗ Errou"); essa confirmação é o que fica registrado no arquivo de testes.

Os três exemplos a seguir foram gerados reproduzindo fielmente, em uma simulação independente, as mesmas funções de inferência do sistema (`melhor_pergunta`, `filtrar_candidatos` e `ranking_candidatos`) sobre a base de conhecimento real. O Exemplo 1 foi conferido contra o registro correspondente em `testes.json`, confirmando que a simulação reflete exatamente o comportamento do sistema em produção.

5.1 Exemplo 1 — Caso típico, com respostas precisas (Naruto)

O usuário pensa em Naruto e responde com precisão a todas as perguntas. O sistema converge para a resposta correta em apenas 4 perguntas — exatamente o valor registrado para esse personagem em `testes.json`.

Nº	Pergunta	Resposta	Candidatos restantes
1	É protagonista?	Sim	15
2	É estudante?	Não	8
3	É criança ou adolescente?	Sim	3
4	É ninja?	Sim	1 → Naruto

Redução progressiva do espaço de hipóteses: 27 → 15 → 8 → 3 → 1

5.2 Exemplo 2 — Interferência entre personagens semelhantes (Deku, Bakugo e Todoroki)

Este exemplo ilustra uma situação de múltiplas hipóteses permanecendo possíveis ao final do jogo. Uma análise de distância entre os perfis de atributos de todos os pares de personagens da base (seção 6) revela que Deku e Todoroki diferem em apenas 1 dos 20 atributos ("protagonista"), e Bakugo e Todoroki também diferem em apenas 1 atributo ("anti_heroi") — o par e o segundo par mais parecidos de toda a base. No cenário abaixo, o usuário pensa em Todoroki, mas responde "não sei" exatamente às duas perguntas que o distinguiriam de Deku e Bakugo.

Nº	Pergunta	Resposta	Candidatos restantes
1	É protagonista?	Não sei	27
2	É criança ou adolescente?	Sim	12
3	Possui transformação marcante?	Não	6
4	É estudante?	Sim	4
5	É masculino?	Sim	3
6	É um anti-herói?	Não sei	3 (empate)

Esgotados os atributos capazes de discriminar os três candidatos remanescentes, o sistema recorre ao ranqueamento por pontuação, que resulta em um empate técnico:

Posição	Personagem	Pontuação
1º	Deku	4 pontos
2º	Bakugo	4 pontos
3º	Todoroki	4 pontos

Como os três candidatos somam exatamente a mesma pontuação, o critério de desempate do sistema (ordenação estável, que preserva a ordem original de cadastro) escolhe Deku — resultando em uma resposta incorreta do ponto de vista do usuário, que pensava em Todoroki, apesar de o sistema ter operado corretamente segundo sua própria lógica. Esse é precisamente o tipo de situação que o painel "Ver hipóteses atuais" foi projetado para tornar visível ao usuário antes mesmo do palpite final.

5.3 Exemplo 3 — Personagem ausente da base de conhecimento (ilustrativo)

Diferentemente dos dois exemplos anteriores, este é um cenário ilustrativo (não um teste efetivamente registrado em testes.json), construído para expor o comportamento do sistema diante de um personagem que o usuário conhece, mas que não está cadastrado — por exemplo, Itachi Uchiha, do universo de Naruto. Atribuindo a esse personagem hipotético respostas plausíveis para as perguntas da base:

Nº	Pergunta	Resposta	Candidatos restantes
1	É protagonista?	Não	12
2	É criança ou adolescente?	Não	6

Nº	Pergunta	Resposta	Candidatos restantes
3	Possui poderes especiais?	Sim	2
4	O personagem é humano?	Sim	1 → Gojo (incorreto)

O resultado é revelador: o sistema não responde "não sei". Consistentemente com a propriedade demonstrada na seção 4.4, ele converge com total confiança para Gojo — um personagem real da base, mas incorreto do ponto de vista do usuário — simplesmente porque Gojo foi o único personagem cadastrado compatível com as quatro respostas dadas. Esse resultado é retomado com mais profundidade na análise crítica da seção 6.

6. Análise dos Resultados Obtidos

6.1 Metodologia dos experimentos automáticos

A aba "📊 Testes" da aplicação executa, sob demanda, um experimento automático completo: para cada um dos 27 personagens cadastrados na base, o sistema simula uma partida inteira respondendo, a cada pergunta, exatamente o valor real armazenado para aquele personagem — como se um usuário perfeitamente informado estivesse jogando. O resultado de cada partida simulada (acerto ou falha, personagem pensado, personagem chutado e número de perguntas necessárias) é registrado em testes.json. Trata-se de um experimento de autovalidação interna do algoritmo — controlado e sem ruído —, que mede a eficiência do mecanismo de busca, mas não substitui testes com usuários humanos reais, cujas respostas podem ser imprecisas, incertas ou referir-se a personagens fora da base.

6.2 Número médio de perguntas e taxa de acerto

Os 27 testes automáticos registrados cobrem a totalidade da base de conhecimento. O sistema obteve 27 acertos e 0 falhas nesse conjunto — uma taxa de acerto de 100,0% —, com uma média de 4,85 perguntas por identificação (desvio padrão de 0,52), mediana e moda de 5 perguntas, mínimo de 4 e máximo de 6 perguntas.

Perguntas até a identificação	Nº de personagens (testes)
4	6
5	19
6	2

Distribuição do número de perguntas nos 27 testes automáticos (média = 4,85)

Esse resultado é, na prática, muito próximo do limite teórico: para distinguir 27 hipóteses apenas com perguntas de resposta binária, o número mínimo de perguntas necessário no melhor caso possível (partições perfeitamente equilibradas a cada passo) é $\log_2(27) \approx 4,75$, arredondado para 5 perguntas no pior caso. A média observada de 4,85 perguntas fica a menos de 0,1 pergunta acima desse limite teórico — evidência de que a heurística gulosa implementada em `melhor_pergunta()`, apesar de simplificada, aproxima-se consistentemente do desempenho ótimo de uma busca binária bem balanceada sobre esta base específica.

É importante frisar o que esse número realmente mede: como cada teste alimenta o sistema com as respostas verdadeiras e completas do próprio personagem-alvo, a taxa de acerto de 100,0% reflete a corretude e a eficiência do algoritmo de busca em condições ideais — não a taxa de acerto que se deve esperar de usuários humanos reais, que podem responder "não sei", hesitar, errar uma resposta ou, mais criticamente, pensar em um personagem que simplesmente não existe na base.

6.3 Casos de falha: personagens fora da base de conhecimento

Como demonstrado no Exemplo 3 (seção 5) e fundamentado na propriedade descrita na seção 4.4, o sistema nunca responde "não consegui identificar" enquanto a base de conhecimento não estiver vazia — mesmo quando o personagem que o usuário tem em mente jamais foi cadastrado. Nessas situações, o sistema converge com aparente confiança para o personagem cadastrado que mais se aproxima das respostas dadas, o que só é percebido como falha quando o usuário confirma manualmente que o palpite estava incorreto (botão "✗ Errou"). Isso implica que a taxa real de falha por ausência de personagens na base não pode ser medida a partir de testes.json — cujos 27 registros cobrem, por construção, apenas personagens já cadastrados —, sendo

necessário testar o sistema com usuários que deliberadamente pensem em personagens fora da base para quantificar esse tipo de falha (ver seção 6.5, trabalhos futuros).

6.4 Casos de falha: interferência entre personagens semelhantes

A segunda fonte relevante de falha ocorre quando dois ou mais personagens cadastrados têm perfis de atributos quase idênticos. Para quantificar esse risco, calculou-se a distância entre os perfis de atributos de todos os 351 pares possíveis de personagens da base (número de atributos, dentre os 20, em que os dois personagens divergem). A Tabela 4 lista os pares mais parecidos.

Personagem A	Personagem B	Atributos divergentes
Deku	Todoroki	1
Bakugo	Todoroki	1
Naruto	Gon	2
Goku	Vegeta	2
Zoro	Levi	2
Deku	Bakugo	2
Deku	Yuji	2
Deku	Saitama	2

Tabela 4 — Pares de personagens mais parecidos na base (menor distância = maior risco de interferência)

Os dois pares mais parecidos — Deku/Todoroki e Bakugo/Todoroki, ambos divergindo em apenas 1 dos 20 atributos — formam justamente o trio explorado no Exemplo 2 (seção 5), no qual uma resposta "não sei" às duas perguntas discriminantes é suficiente para gerar um empate de três hipóteses. Esse tipo de interferência é uma consequência esperada de se usar um número relativamente pequeno de atributos amplos (20, no mínimo estabelecido pelo enunciado) para diferenciar personagens de um mesmo arquétipo dentro da mesma obra — como os estudantes-heróis de "My Hero Academia". É precisamente para tornar esse tipo de situação visível ao usuário, em vez de escondê-la atrás de um palpite aparentemente confiante, que o sistema disponibiliza o painel "Ver hipóteses atuais" com o aviso de empate descrito na seção 5.

6.5 Síntese e limitações

Em síntese, os experimentos automáticos confirmam que o mecanismo de inferência descrito na seção 4 — busca em espaço de hipóteses combinada a regras de eliminação e a um critério de pontuação para desempate — é eficiente e correto sobre a base atual, identificando qualquer personagem cadastrado em, no máximo, 6 perguntas e, em média, muito próximo do limite teórico de $\log_2(27)$ perguntas. Ao mesmo tempo, a análise qualitativa revela que as duas fragilidades reais do sistema não são de eficiência, mas de cobertura e de resolução:

- Personagens fora da base nunca são rejeitados — são silenciosamente confundidos com o cadastro mais parecido;
- Personagens muito semelhantes dentro da própria base, sobretudo dentro de uma mesma obra, produzem empates que dependem de o usuário responder com certeza às poucas perguntas realmente discriminantes entre eles.

Como trabalhos futuros, destacam-se:

- A realização de testes com usuários humanos reais e diversos, para medir a taxa de acerto e o número médio de perguntas em condições reais de incerteza — o que o experimento atual, por ser uma autovalidação com respostas perfeitas, não é capaz de capturar;
- A inclusão de atributos adicionais mais discriminantes para separar os pares mais parecidos identificados na Tabela 4;
- A adoção de uma pontuação probabilística (por exemplo, de inspiração bayesiana) no lugar da contagem simples de acertos, de modo a comunicar ao usuário um nível de confiança explícito mesmo quando resta um único candidato, em vez de tratar toda hipótese final como igualmente certa.

7. Considerações Finais

O sistema desenvolvido atende e supera todos os requisitos mínimos estabelecidos para a base de conhecimento: 27 entidades cadastradas (35% acima do mínimo de 20) e 20 atributos (33% acima do mínimo de 15), com uma representação explícita e consistente do conhecimento. O mecanismo de inferência híbrido — busca em espaço de hipóteses, eliminação por regras e pontuação para desempate — mostrou-se, nos experimentos automáticos, eficiente e próximo do limite teórico de perguntas necessário para distinguir entre os personagens cadastrados, ao mesmo tempo em que a interação por Sim/Não/Não sei, com exibição das hipóteses atuais, cumpre integralmente os requisitos de interação com o usuário.

Mais importante do que os números de eficiência, porém, é a compreensão qualitativa obtida sobre as reais limitações do sistema: sua fragilidade não está em identificar personagens já conhecidos — nisso ele é consistentemente rápido e correto —, mas em lidar com o que está fora ou nas bordas da sua base de conhecimento, seja um personagem nunca cadastrado, seja um par de personagens quase indistinguíveis entre si. Reconhecer e caracterizar precisamente essas duas situações, como feito na seção 6, é o principal resultado analítico deste trabalho, e aponta diretamente o caminho para as evoluções futuras do sistema.